W13D4 – Exploit DVWA: XSS e SQL Injection

Studente: Riccardo Luzzi

Corso: Cyber Security

Sistema utilizzato: Kali Linux

Strumenti utilizzati: DVWA, Burp Suite Community Edition, SQLmap, Chromium, VirtualBox, pfSense

Target: Metasploitable – 192.168.51.101

Scenario di rete: Kali 192.168.50.100 e Metasploitable 192.168.51.101 su reti differenti, instradate tramite pfSense

Introduzione

L’esercitazione W13D4 riguardava due vulnerabilità presenti in DVWA: XSS Reflected e SQL Injection non blind. Ho utilizzato lo stesso laboratorio del precedente esercizio, con Kali e Metasploitable su due reti separate collegate tramite pfSense. Dopo aver controllato la comunicazione tra le macchine ho impostato DVWA sul livello Low e ho iniziato le prove dal browser.

Per XSS ho verificato prima l’interpretazione dei tag HTML, poi l’esecuzione di JavaScript e la lettura dei cookie accessibili alla pagina. Per SQL Injection sono partito da una ricerca normale, ho provocato un errore nella query e ho utilizzato UNION SELECT per interrogare il database. Ho svolto anche la parte facoltativa sui livelli Medium e High e, come controllo aggiuntivo, ho ripetuto alcune verifiche con SQLmap da terminale.

1. Verifica del laboratorio

Su Kali ho controllato l'indirizzo di rete, risultato 192.168.50.100, e ho inviato quattro pacchetti ICMP a 192.168.51.101. Tutte le risposte sono arrivate senza perdite. Ho quindi aperto DVWA nel browser e selezionato il livello di sicurezza Low.

Figura 1 – Indirizzo della Kali e ping verso Metasploitable

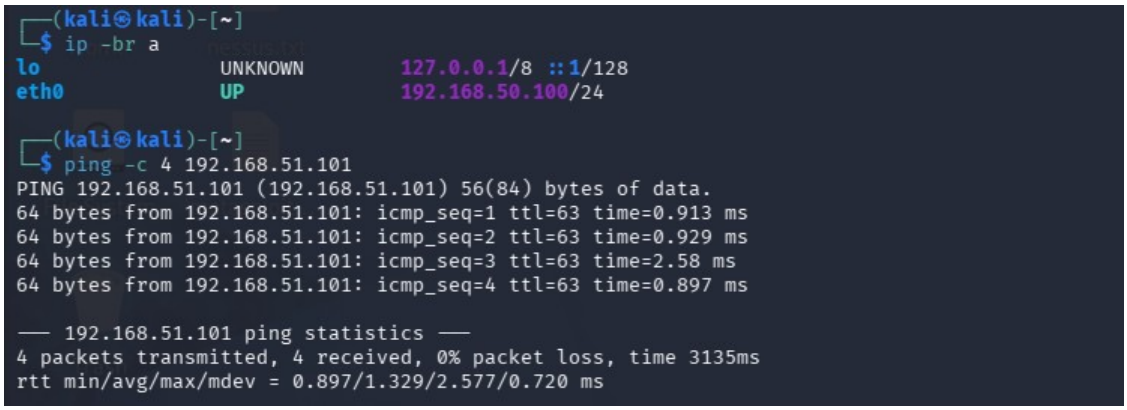
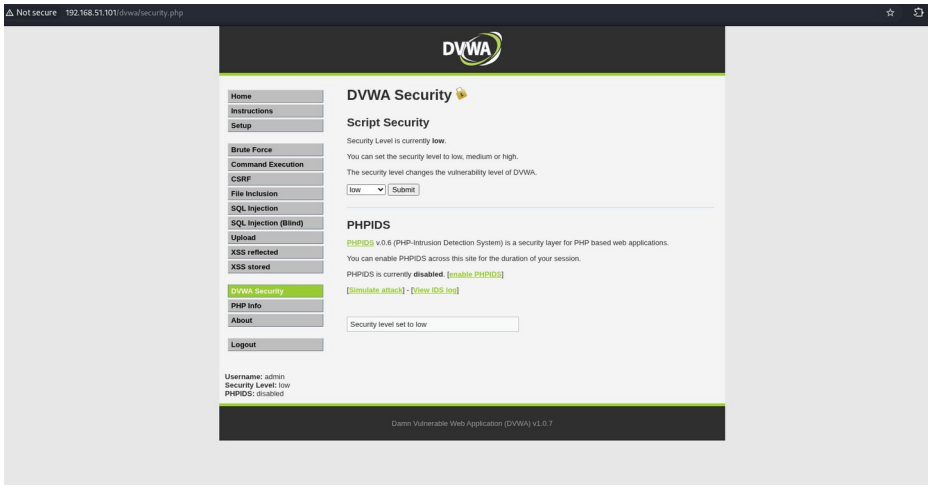


Figura 2 – DVWA impostato su Low



2. XSS Reflected – Security Level Low

2.1 Interpretazione HTML ed esecuzione JavaScript

Nella pagina XSS Reflected ho provato inizialmente il tag Riccardo. La risposta “Hello Riccardo” ha mostrato il nome in corsivo: l’input non veniva trattato soltanto come testo, ma interpretato come HTML. Successivamente ho inserito uno script con alert(). Questa volta il browser ha aperto una finestra con il messaggio scelto, confermando l’esecuzione di JavaScript.

<script>alert('Test di sicurezza: vulnerabilita XSS rilevata!')</script>

Figura 3 – Tag HTML interpretato nella risposta XSS

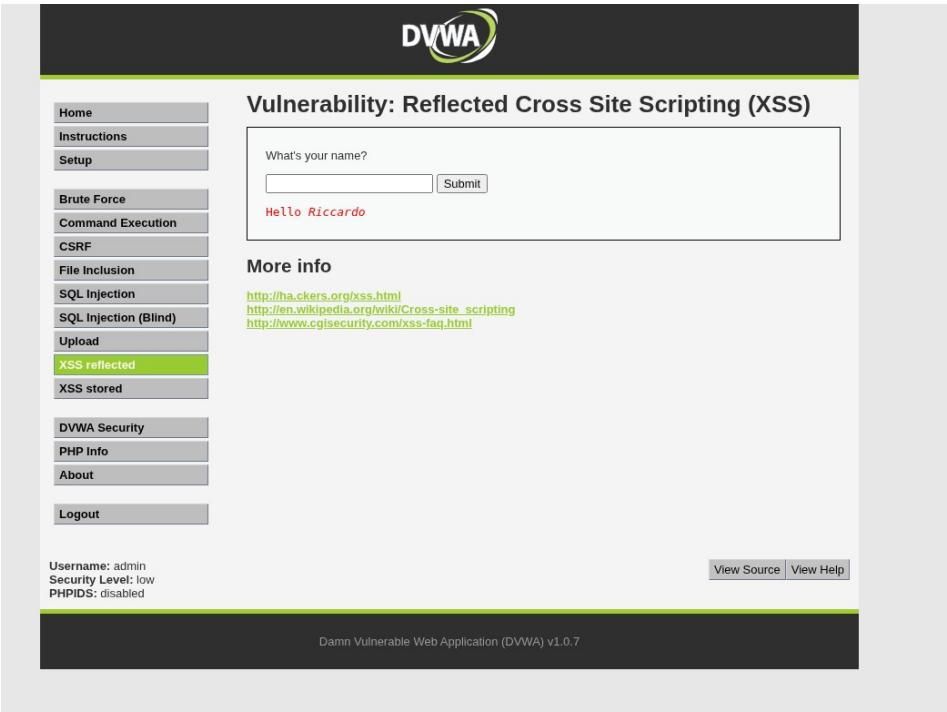
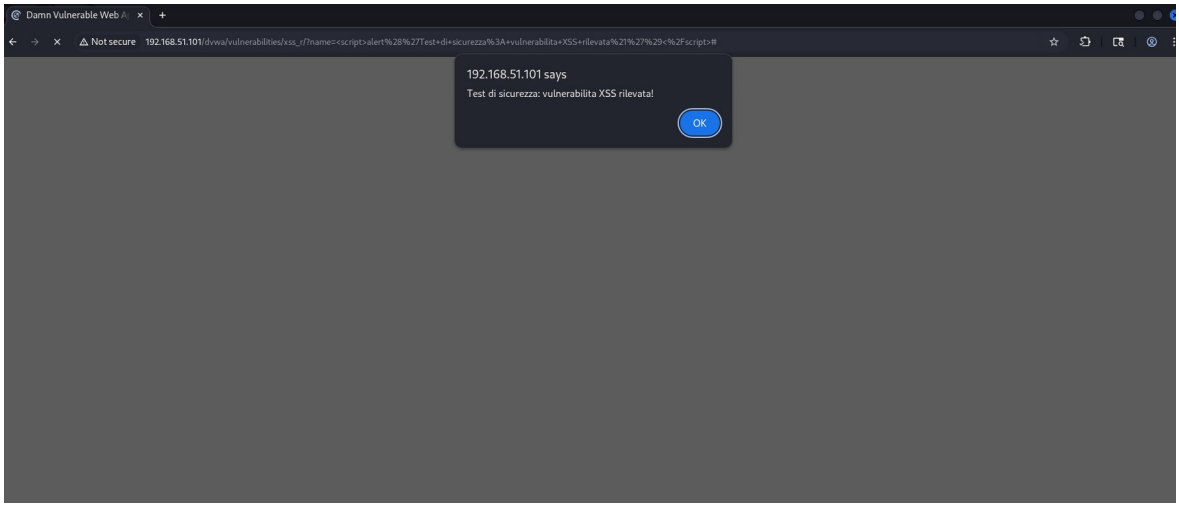


Figura 4 – Alert JavaScript eseguito sul livello Low



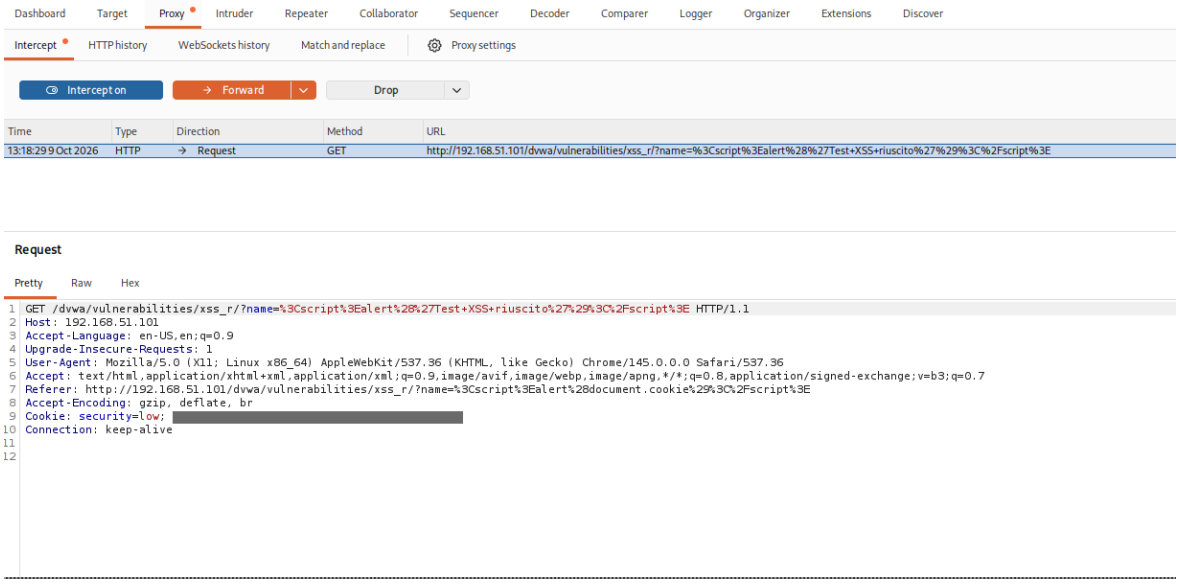
2.2 Cookie di sessione e richiesta HTTP

Ho poi utilizzato document.cookie in un alert per vedere quali cookie fossero leggibili da JavaScript. La pagina mostrava security=low e PHPSESSID. Questo secondo valore identifica la sessione PHP; la prova dimostra che, in questa configurazione, il cookie era accessibile allo script e non protetto da HttpOnly. Non ho inviato il cookie verso alcun server esterno.

```
<script>alert(document.cookie)</script>
```

Per controllare anche il traffico HTTP, ho intercettato con Burp Suite una richiesta GET alla pagina xss_r. Nella query string era visibile il parametro name con lo script codificato per l'URL. Ho oscurato il cookie di sessione prima di inserire l'immagine nel report.

Figura 5 – Burp Suite: richiesta GET con payload XSS e sessione oscurata

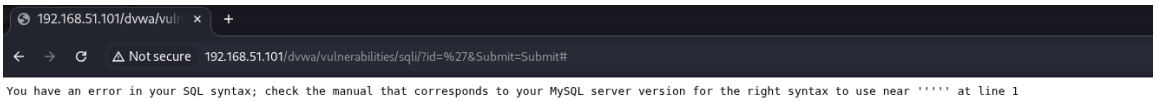


3. SQL Injection – Security Level Low

3.1 Verifica iniziale e errore SQL

Dalla pagina SQL Injection ho inserito l'ID 1, ottenendo i dati dell'utente admin. Come primo controllo di iniezione ho poi inserito un apostrofo singolo. DVWA ha mostrato un errore di sintassi MySQL, segnale che il carattere immesso interferiva con la query costruita sul server.

Figura 6 – Errore SQL provocato con il carattere apostrofo



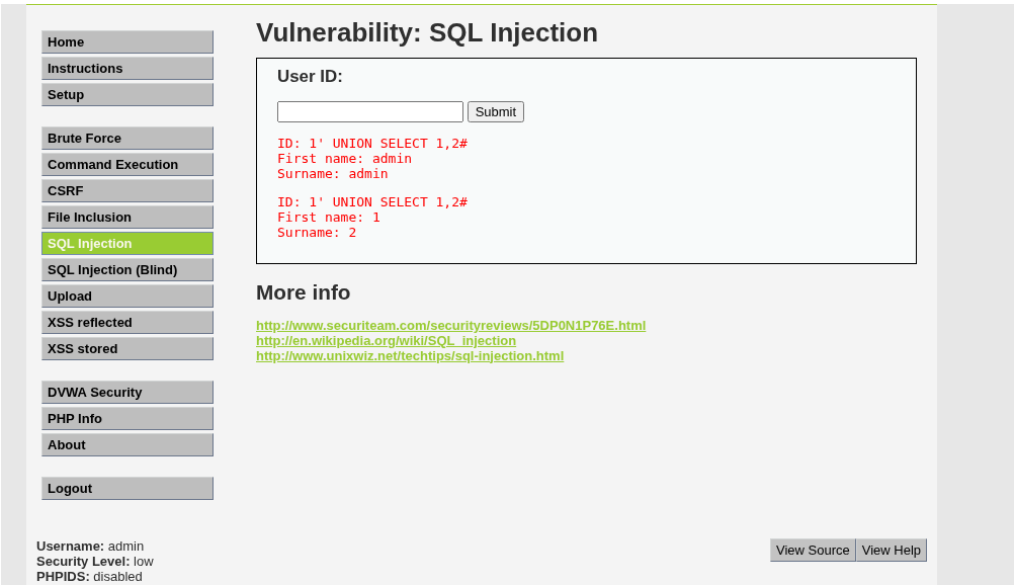
3.2 UNION SELECT e informazioni sul database

Ho utilizzato UNION SELECT per aggiungere una seconda SELECT a quella originale. Con i valori 1 e 2 ho verificato che la query restituiva due colonne visibili, visualizzate nelle posizioni First name e Surname. In seguito le ho sostituite con database() e version(), ottenendo il nome dvwa e la versione MySQL 5.0.51a-3ubuntu5.

Comando SQL:

1' UNION SELECT 1,2#

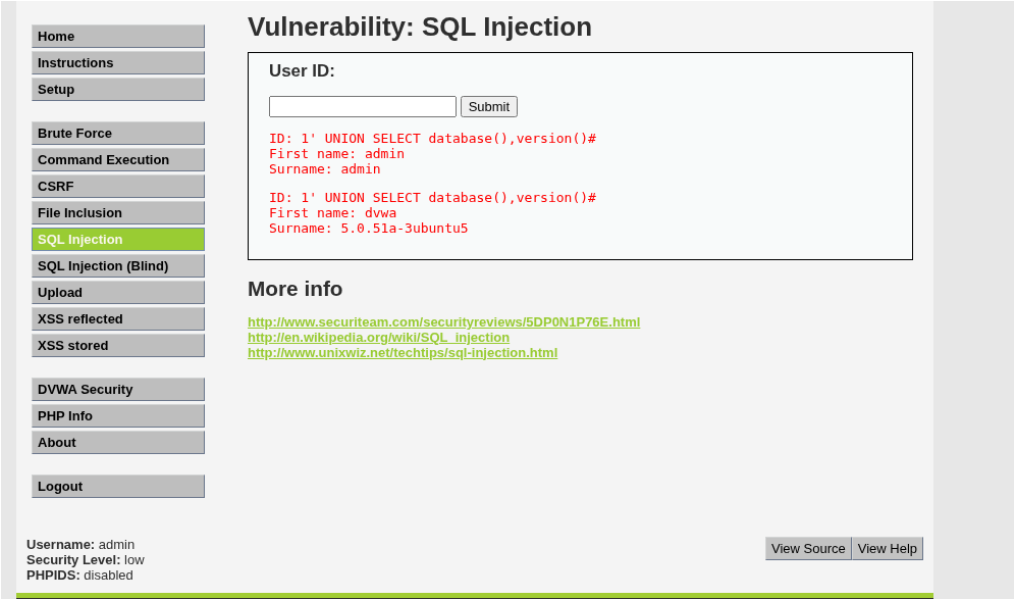
Figura 7 – UNION SELECT con due valori di prova



Comando SQL:

1' UNION SELECT database(),version()#

Figura 8 – Nome del database e versione MySQL restituiti dalla UNION



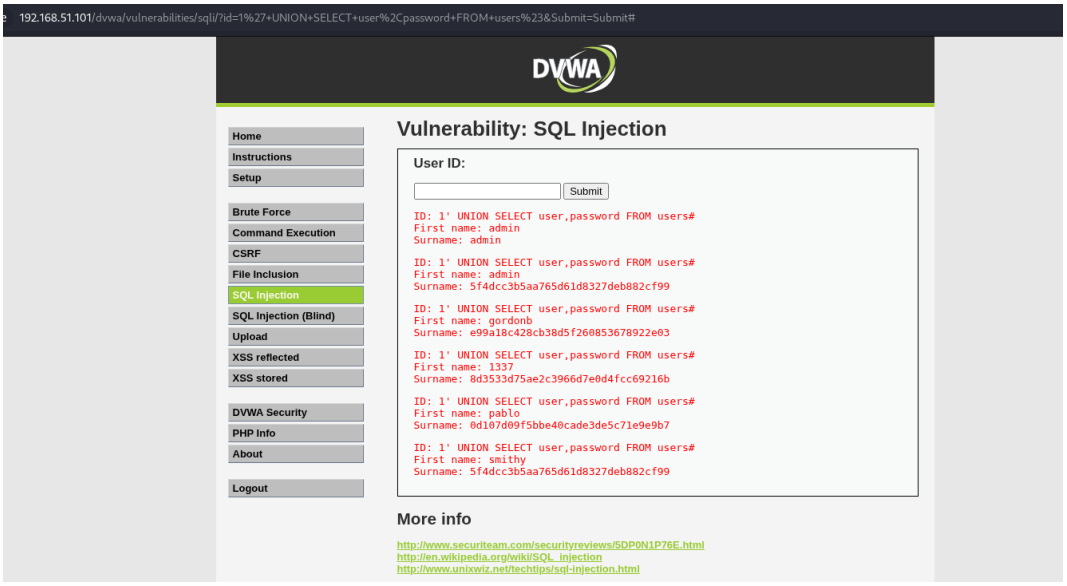
3.3 Tabella users e struttura del database

Come approfondimento ho interrogato la tabella users e ho ottenuto cinque username con i rispettivi hash delle password. Non si trattava di password in chiaro: a questo punto l'estrazione manuale mostrava i valori memorizzati nel database. Ho inoltre consultato information_schema.tables, individuando le tabelle guestbook e users nello schema dvwa.

Comando SQL:

1' UNION SELECT user,password FROM users#

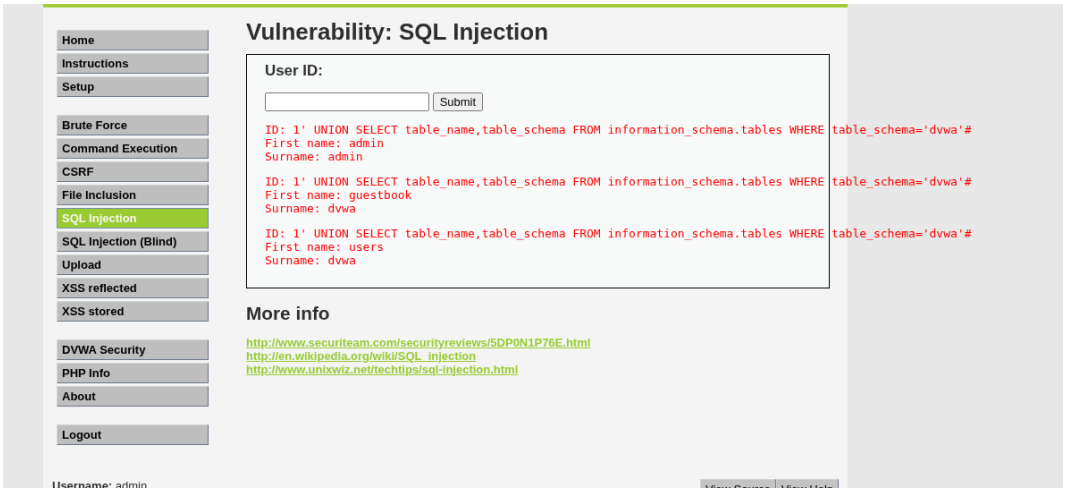
Figura 9 – Dati della tabella users recuperati con UNION SELECT



Comando SQL:

1' UNION SELECT table_name,table_schema FROM information_schema.tables WHERE table_schema='dvwa'#

Figura 10 – Elenco delle tabelle guestbook e users nello schema dvwa



4. Parte facoltativa – SQL Injection Medium e High

4.1 Medium: tentativo iniziale e correzione

Su Medium ho provato inizialmente lo stesso payload di Low con l'apostrofo, ma la pagina ha restituito un errore. Dal View Source ho visto che DVWA applicava mysql_real_escape_string() all'ID. Il valore, però, veniva inserito nella query senza apostrofi: per questo ho ripetuto il test usando UNION SELECT senza il carattere iniziale di chiusura della stringa.

```
1 UNION SELECT database(),version()#
```

Figura 11 – Sorgente SQL Injection Medium: escaping e parametro numerico non quotato

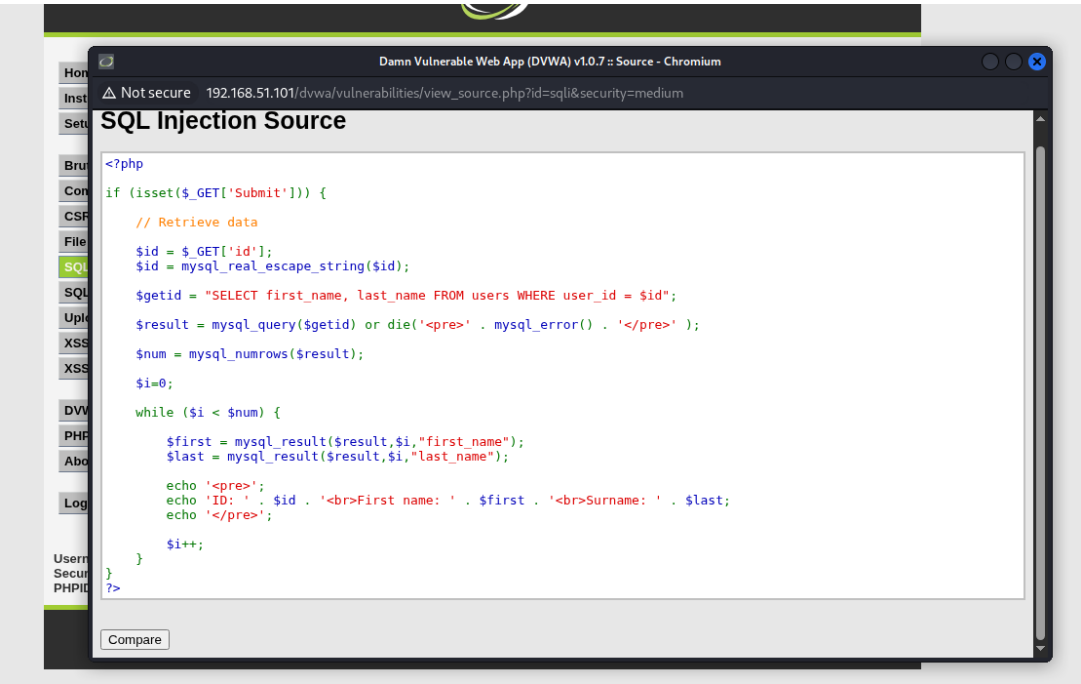
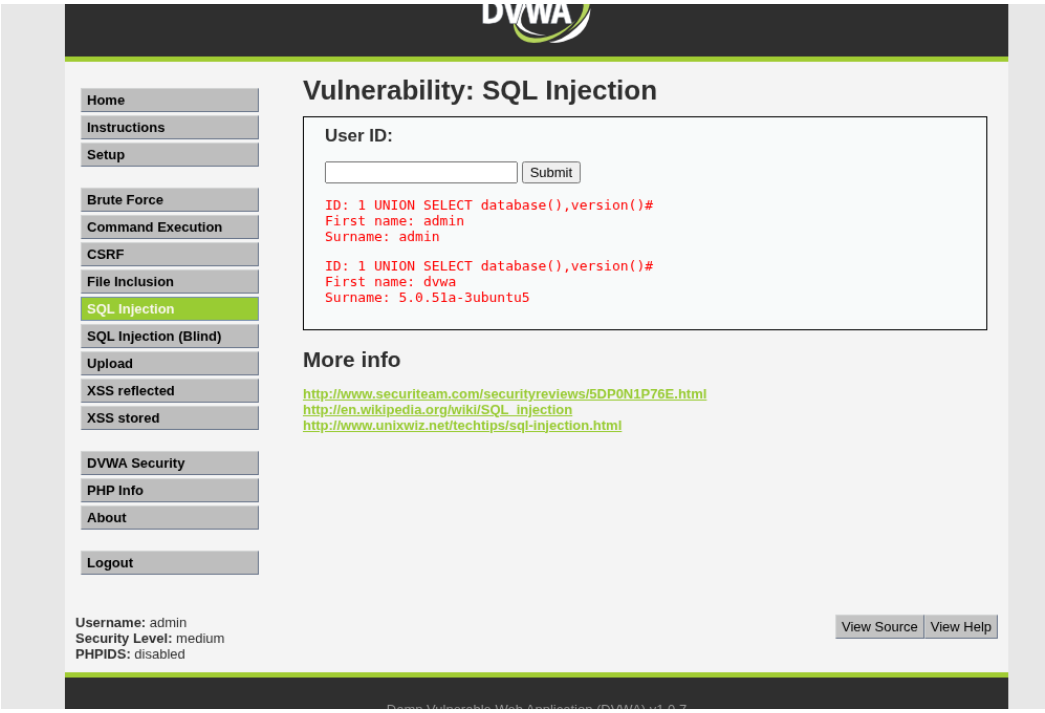


Figura 12 – UNION SELECT riuscita sul livello Medium



4.2 High: controllo sul valore numerico

Nel codice del livello High ho trovato anche stripslashes() e is_numeric(). La query viene eseguita solo quando il valore ID supera il controllo numerico. Infatti con ID 1 è tornato admin, mentre con UNION SELECT la pagina non ha restituito righe. Questo non prova l'assenza di ogni possibile vulnerabilità, ma conferma che il payload impiegato durante il laboratorio viene bloccato dal controllo.

Figura 13 – Sorgente SQL Injection High con controllo is_numeric()

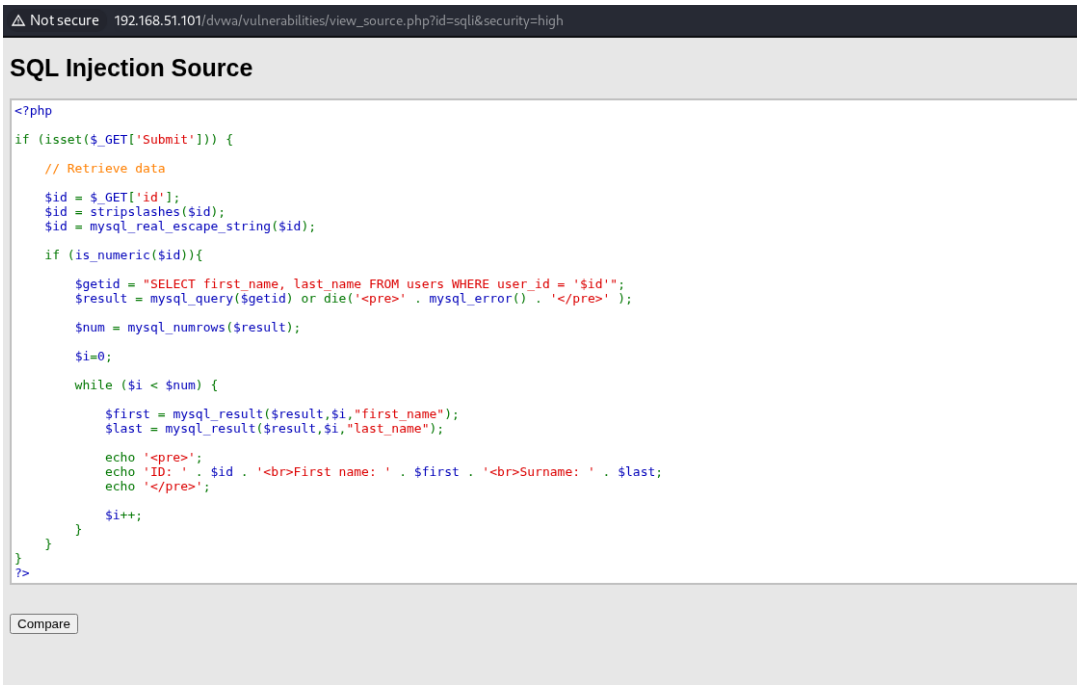
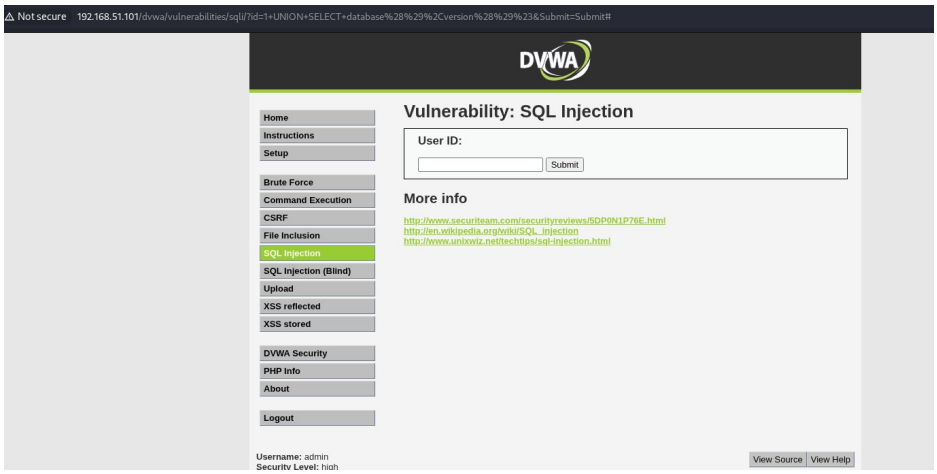


Figura 14 – Tentativo UNION SELECT senza risultati al livello High



5. Parte facoltativa – XSS Reflected Medium e High

5.1 Medium: filtro superato cambiando le maiuscole

Il View Source del livello Medium mostra str_replace("<script>", "", ...). Ho provato prima il tag minuscolo: DVWA ha rimosso l'apertura <script> e nella pagina è rimasto soltanto il testo dell'alert. Ho poi scritto il nome del tag con maiuscole e minuscole alternate. Il filtro non lo ha riconosciuto, mentre il browser lo ha interpretato regolarmente e ha aperto l'avviso.

Comando su XSS reflected:

<ScRiPt>alert('Test XSS Medium riuscito')</ScRiPt>

Figura 15 – Sorgente XSS Medium: sostituzione della sola stringa <script>

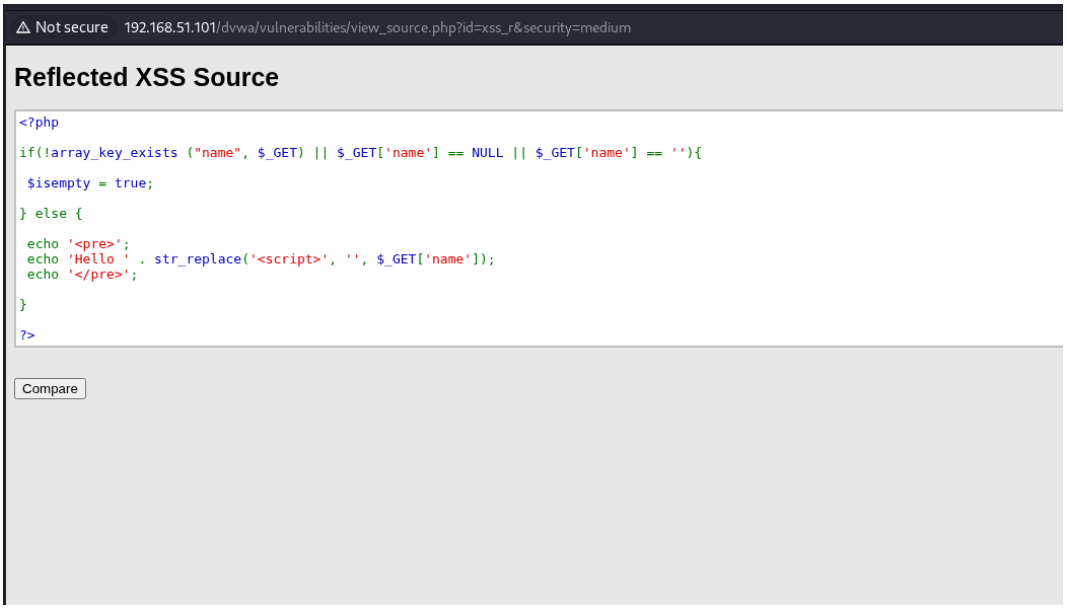


Figura 16 – Tentativo XSS minuscolo: tag rimosso al livello Medium

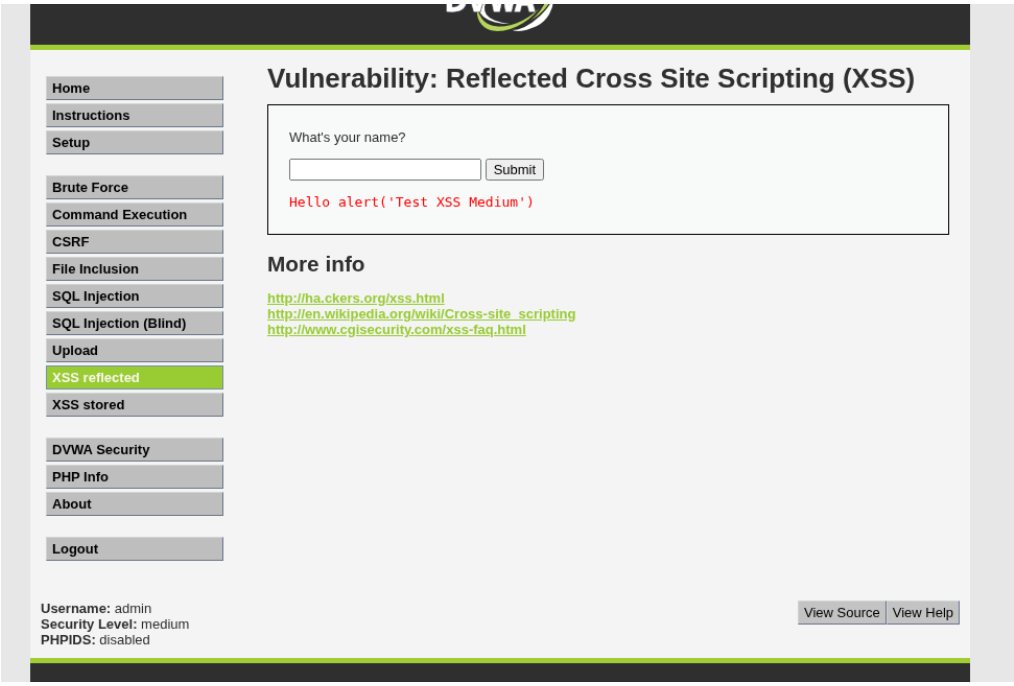
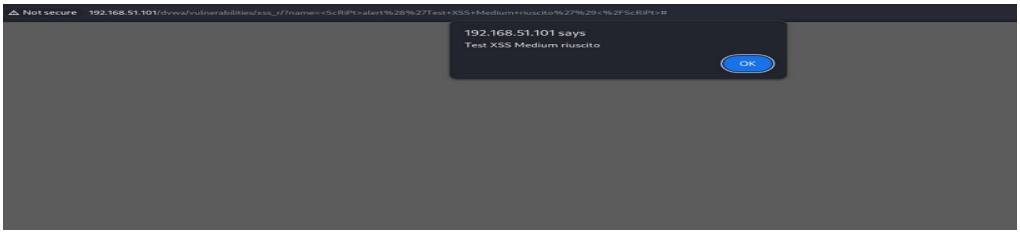


Figura 17 – Bypass Medium con tag ScRiPt: alert eseguito



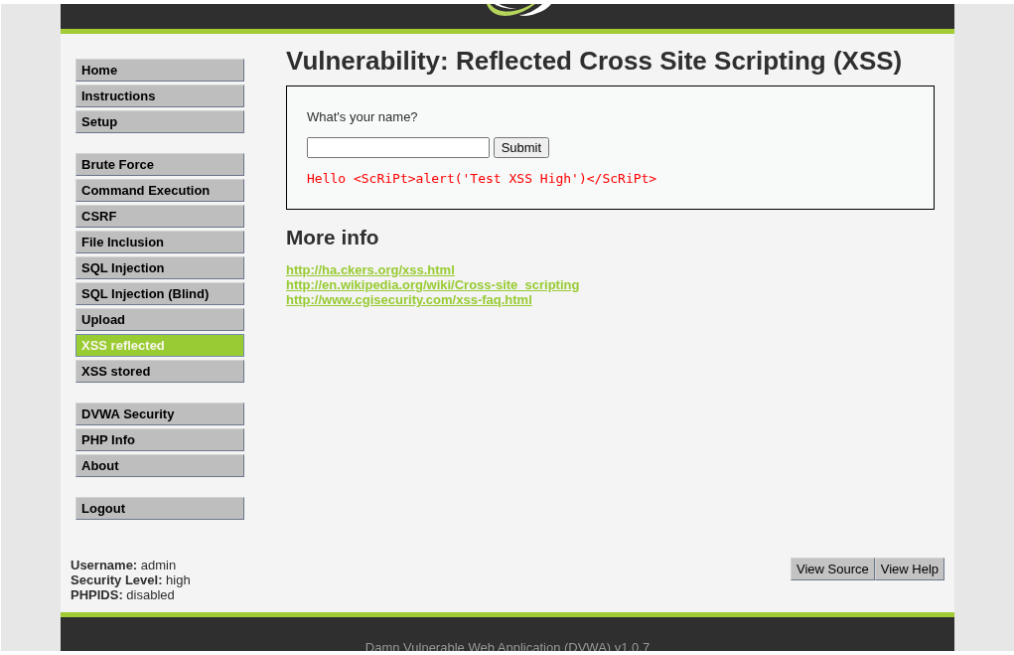
5.2 High: codifica dell'output

Sul livello High la funzione htmlspecialchars() converte i caratteri HTML speciali prima di restituirli nella pagina. Ripetendo lo stesso test, il tag misto è comparso come testo e non è stato eseguito. La differenza con Medium è netta: rimuovere una specifica stringa non equivale a codificare correttamente l'output.

Figura 18 – Sorgente XSS High con htmlspecialchars()



Figura 19 – XSS High: payload visualizzato come testo

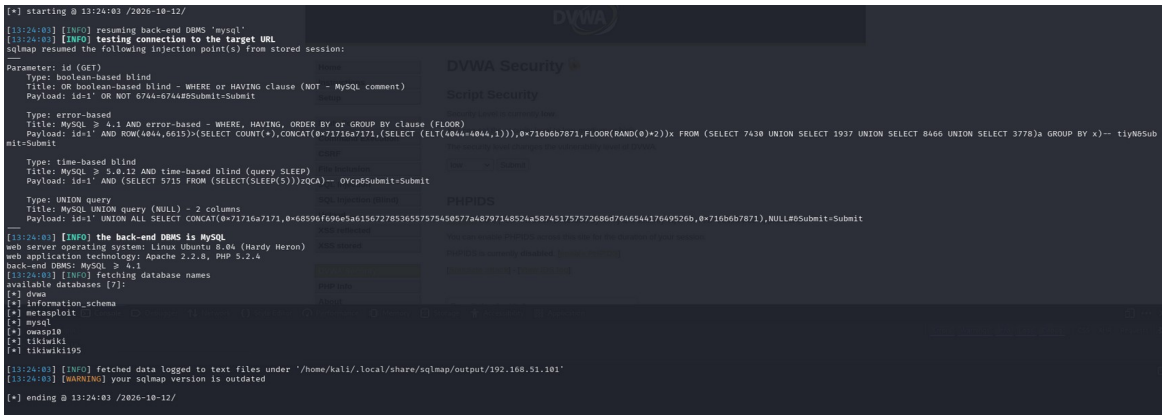


6. Verifica automatizzata con SQLmap

Per completare il confronto ho utilizzato SQLmap dal terminale di Kali, passando il cookie della sessione DVWA autenticata e limitando le prove al target del laboratorio. Con --dbs lo strumento ha identificato il parametro GET id come vulnerabile e ha rilevato tecniche boolean-based, error-based, time-based e UNION, quest'ultima con due colonne. Ha inoltre elencato sette database, tra i quali dvwa.

sqlmap --cookie="security=low; PHPSESSID=[oscurato]" -u "http://192.168.51.101/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" --dbs

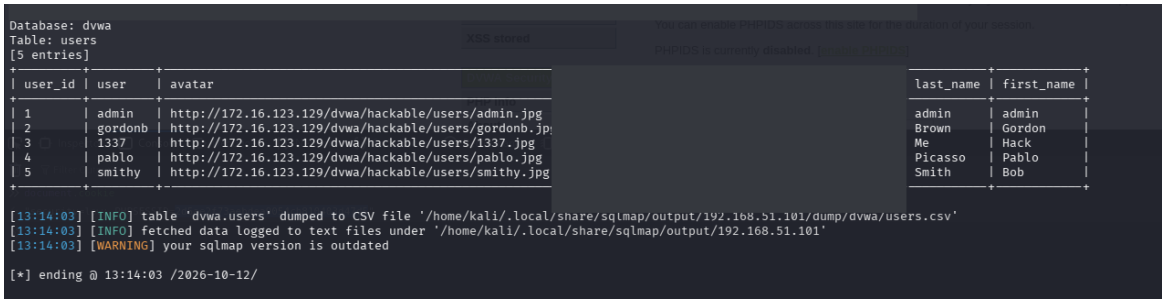
Figura 20 – SQLmap: tecniche rilevate ed elenco dei sette database



Con un secondo comando ho richiesto il dump della sola tabella users del database dvwa. SQLmap ha recuperato cinque record, compresi username, nomi, cognomi e hash. Nello screenshot lo strumento ha anche riconosciuto alcune password corrispondenti agli hash: questo ulteriore passaggio non era presente nella prova manuale. I risultati sono stati salvati in un file CSV locale.

sqlmap --cookie="security=low; PHPSESSID=[oscurato]" -u "http://192.168.51.101/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" -D dvwa -T users --dump

Figura 21 – SQLmap: cinque record estratti dalla tabella users (valori password oscurati)



7. Riepilogo delle prove

Ambito	Low	Medium	High
XSS Reflected	HTML e JavaScript eseguiti; cookie leggibili	Filtro <script> aggirato con ScRiPt	Payload mostrato come testo
SQL Injection	UNION SELECT, metadati e hash ottenuti	UNION senza apostrofo riuscita	Payload UNION bloccato da is_numeric()
SQLmap	7 database e 5 record users	Non eseguito	Non eseguito

8. Comandi e payload principali

Comando / payload	Scopo
ping -c 4 192.168.51.101	Controllo della comunicazione di rete
<script>alert('Test di sicurezza: vulnerabilita XSS rilevata!')</script>	Esecuzione JavaScript con XSS Low
<script>alert(document.cookie)</script>	Controllo cookie accessibili a JavaScript
1' UNION SELECT 1,2#	Verifica delle due colonne in SQL Injection Low
1' UNION SELECT database(),version()#	Lettura di database e versione MySQL
1 UNION SELECT database(),version()#	Bypass della query Medium senza apostrofo
<ScRiPt>...</ScRiPt>	Bypass del filtro XSS Medium
sqlmap ... --dbs	Enumerazione automatizzata dei database
sqlmap ... -D dvwa -T users --dump	Lettura dei record di prova della tabella users

9. Conclusioni

L’esercitazione mi ha permesso di osservare due problemi diversi: nel caso di XSS l’applicazione restituisce l’input in una pagina senza gestirlo correttamente, mentre nella SQL Injection l’input modifica la query inviata al database. Sul livello Low entrambe le vulnerabilità sono state facilmente riproducibili. Le prove Medium hanno mostrato quanto possano essere deboli i controlli parziali: il filtro XSS dipendeva dalle maiuscole, mentre l’escaping SQL non impediva l’inserimento di una UNION senza apostrofo.

Sul livello High i payload utilizzati non hanno funzionato: htmlspecialchars() ha impedito l’interpretazione del tag HTML, e il controllo numerico ha escluso l’input SQL non numerico. Il confronto con SQLmap è stato utile per ritrovare in modo automatico la stessa vulnerabilità già verificata a mano e osservare quanti dati fossero disponibili. Tutte le prove si riferiscono esclusivamente alle macchine virtuali del laboratorio didattico.